

# W4

BLOCK 1 · CLASS 4 · PRACTICAL

## Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

# Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

# Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.



HOW A TEAM STAYS SANE

# One change, start to finish.

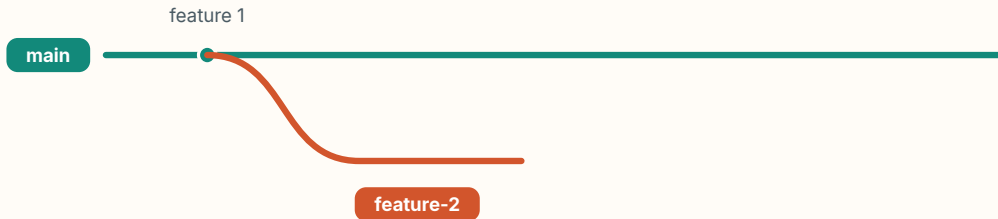
`main` is the version everyone trusts. Here is how a change gets into it.

# main



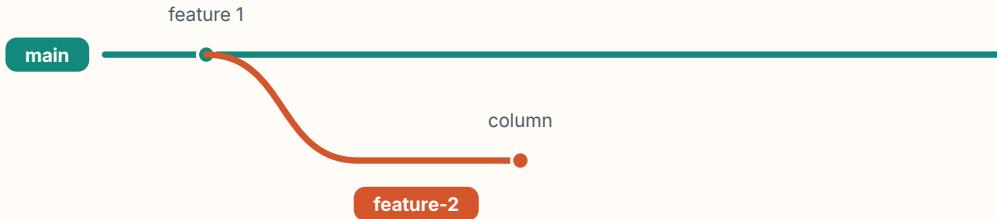
One version, one history, one truth. No folder full of *model\_v3\_FINAL\_jm*.

# A branch.



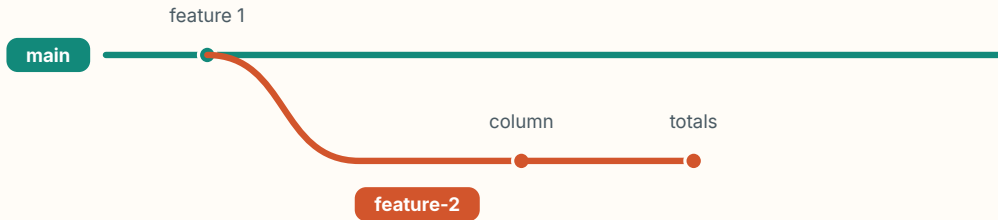
A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

# A commit.



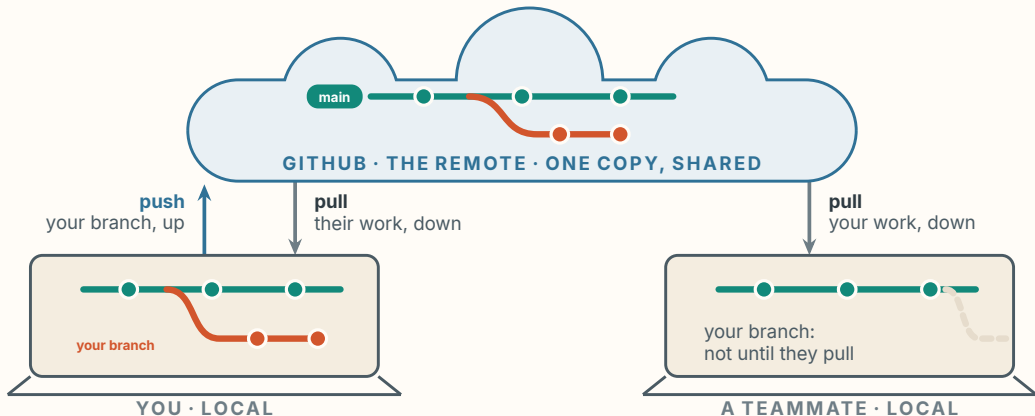
A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

# Working on a branch.



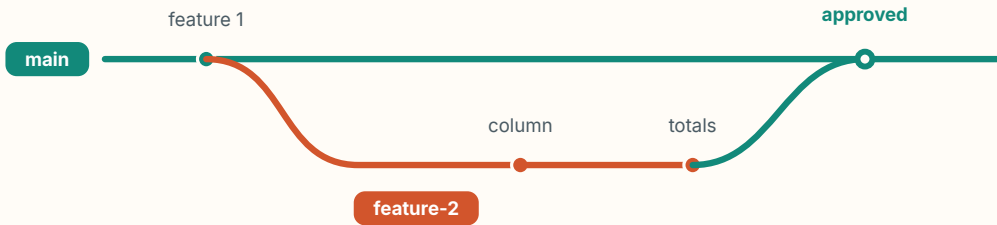
Two commits in and half of it broken. Everyone else's copy is untouched.

# Local, and remote.



Until you **push**, your branch is only on your laptop. **Pull** brings their work down. A **pull request** asks someone to read the diff first — today, you.

# Merge.



**Merge** is the sign-off. **main** now has your work, and the next person starts from there.

# agent

FROM A CHAT MODEL TO AN AGENT

## What is actually running.

A model that can only write text — and the software around it that lets it act.

# The long run-up.



The ideas were all there. What was missing arrived after 2007: **cheap parallel arithmetic** (graphics cards) and **the web as text**.

# Why NVIDIA?



Why is NVIDIA the most valuable company in the world?

One piece of software.

A CPU



a few, fast,  
one thing each

A GPU



thousands, slower,  
all at once

**GAMES · 1999**

Millions of pixels,  
none waiting for  
another. Parallel by  
necessity.



**CUDA · 2007**

Any maths, by any  
programmer, on the  
games chip.



**MINING · 2010s**

The same trick:  
billions of hashes,  
side by side.



**TRAINING · 2012 →**

AlexNet on two  
gaming cards.  
Today: thousands  
of chips, one model.

The chip was built for games. CUDA let everyone else use it — and training a neural network is nothing but parallel arithmetic.

# 2018 onwards.



The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**. The next four slides are that second change.

Bottom row, parameters: Radford et al. (2018, 2019), Brown et al. (2020) · GPT-4: an unconfirmed 2023 report (SemiAnalysis) · none published since

# Ovid's Unicorn.

## A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

## GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

# Guess the next word.

## A PAGE FROM THE WEB

The scientist named the population, after their distinctive \_\_\_\_\_

## THE PAGES

WebText

8 million pages

40 GB

every page a Reddit user had linked to, in 2017. GPT-3, a year later, read Common Crawl: a filtered copy of the public web.



## ITS GUESS

horn 62%  
colour 14%  
feature 9%  
size 5%  
name 3%

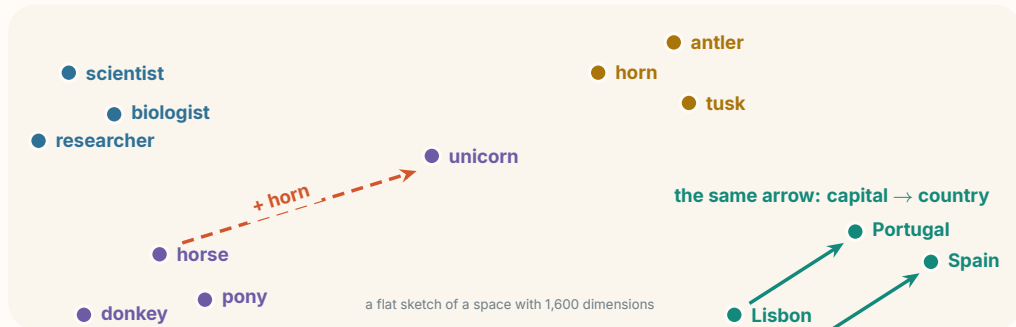
**off by a bit?  
nudge the numbers**

then the next word,  
a few billion times over

That is all training is: hide the next word, guess, get corrected. No grammar lesson, no facts typed in — just 1.5 billion numbers, nudged a few billion times.

Radford et al. (2019): WebText, 8 million pages, 40 GB · Brown et al. (2020): GPT-3 on Common Crawl, filtered · percentages illustrative

# Words become places.

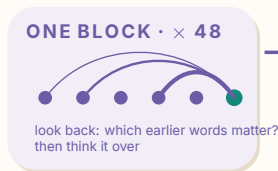
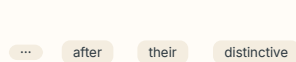


Encode: a word becomes a position. Decode: a position becomes the nearest word. Neighbours mean similar things; directions mean relations.

Positions illustrative · arrow arithmetic: Mikolov et al. (2013), king — man + woman  $\approx$  queen

# Inside the box.

## WORDS IN



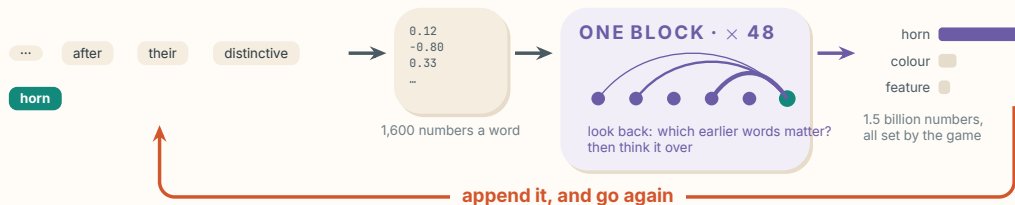
## NEXT WORD



Words become numbers. One block, repeated 48 times, lets each word look back at the ones before it and think it over. Out comes the guess.

# One word at a time.

## WORDS IN



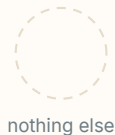
## SO THE STORY GROWS

...distinctive horn . These four -horned , silver -white unicorns

each word chosen from the guess, then fed back in; nobody planned the sentence

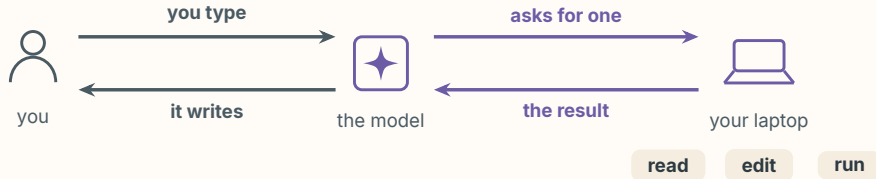
Pick a word from the guess, append it, run the whole box again. The story grows one word at a time. Nobody planned the sentence — and nothing checks it.

# A language model.



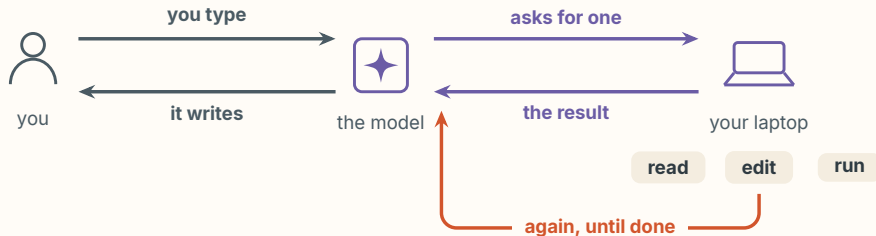
Text in, text out. It has never seen your folder and cannot run anything. In Class 2, **you** were the hands.

# Tools.



A **tool** is a named action it may ask for. Three cover nearly everything. The result comes back to **it**, not to **you**.

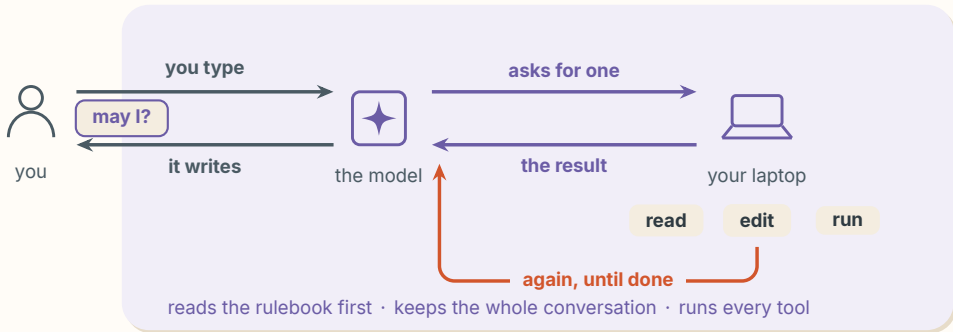
# An agent.



**Agent = model + tools + a loop.** It acts, sees the result, and chooses again.

# The harness.

## CODEX · THE HARNESS



The software around the model. Codex is one; Claude Code and Cursor are others. Same loop everywhere — **learn it once**.

# One exchange, in full.

**YOU TYPE** what was the last expense?

**THE MODEL EMITS** `{"tool": "read_file", "path": "data/expenses.csv"}`

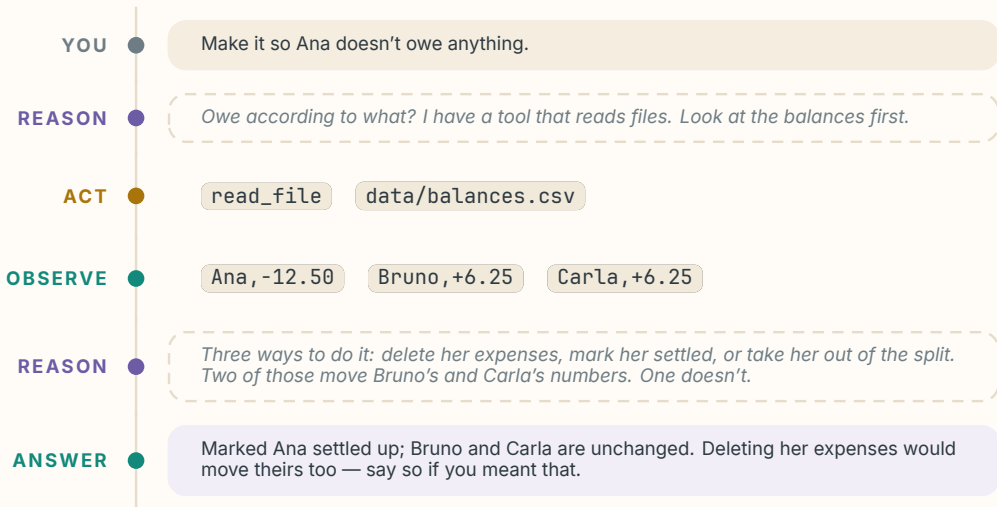
**CODEX ASKS** *Codex wants to read* `data/expenses.csv` *. Allow?*

**CODEX PASTES BACK** `29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25`

**THE MODEL ANSWERS** €30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

# Reasoning.



# The loop has a name: ReAct.



**Reason, act, observe, repeat.** Every harness since is this loop with better plumbing.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · [arxiv.org/abs/2210.03629](https://arxiv.org/abs/2210.03629)

# Context.

## ONE PAGE · EVERYTHING IT CAN SEE

### THERE BEFORE THE FIRST THOUGHT

TOOLS

read a file · edit lines · run a command

RULEBOOK

Run the tests before saying done.

YOU

Bug 002: removing a member crashes the app.

### ADDED AS IT WORKS, AT THE BOTTOM

IT THINKS

Where is a member removed? I have a tool that reads files.

CALL

read the file that holds the logic

RESULT

the whole file, 140 lines, now on the page

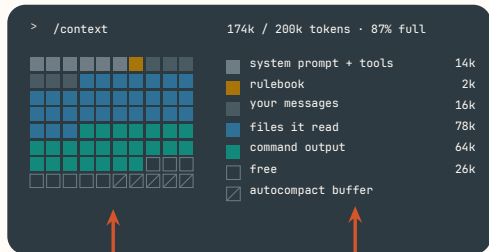


**the tool's own line is on the page —  
that is how it knows which one to call**

Nothing that is not on  
the page exists for it.

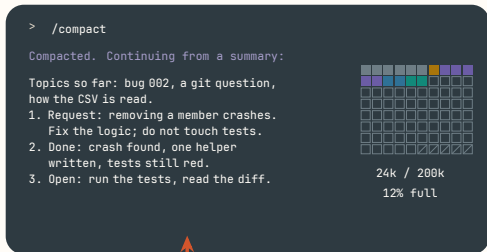
# Context rot.

## AN HOUR IN



an hour of files, outputs and three unrelated questions, all still in there

here it compacts on its own — or you ask first



the conversation folded into a short summary; the rules and the last turns kept

A full, messy window makes the work worse. Compaction clears it.

Measured by Liu et al. (2024) and Chroma (2025); both linked on the week page.

# 3

THE CORE OF THE CLASS

## Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

## AGENTS.md

### WHAT IT IS

#### **A README for agents.**

Every session starts by reading it.

### WHAT GOES IN

#### **What it cannot guess.**

Commands. Boundaries.  
How to report back.

### WHAT STAYS OUT

#### **What it can read.**

Your code, explained.  
Every extra line hides a real one.

It lives in git. When the agent misbehaves, the fix is usually a line here.

# Three rules.

## BOUNDARY

**One branch per spec.**

Never straight to `main`.

## EVIDENCE

**Show me the test output.**

The output itself, not a summary of it.

## OFF-LIMITS

**Never touch the tests.**

If one looks wrong, say so and stop.

Exact wording on the Week 4 page. Add them, read the diff, commit, push.

# The judge.

## THE PROBLEM

**It stops when it looks done.**

So give it a check: failing tests are the to-do list, green is the finish line.

## THE RULE

**Editing a test is cheating.**

It is not finishing.

## EVIDENCE

**Show me the output.**

Anyone can type "all tests pass".

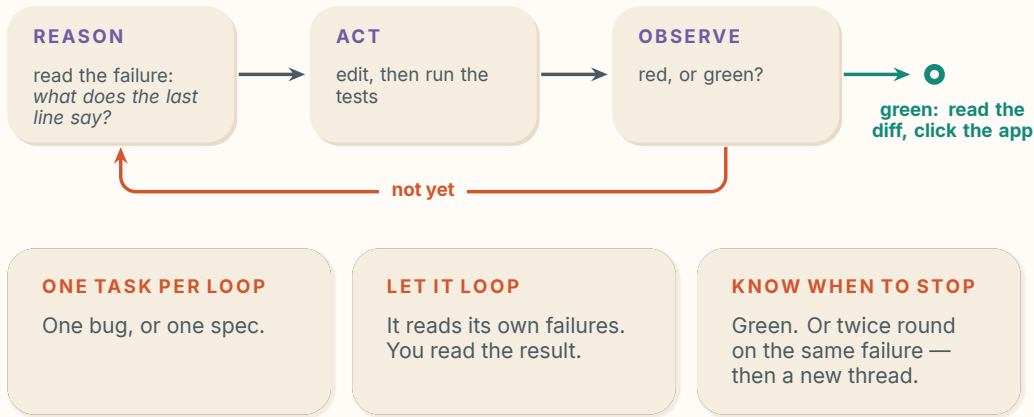
## AFTER GREEN

**Read the diff. Click the app.**

App wrong, tests green? You found a missing test.

Decide what *done* means before you start.

# The loop.



# The shape of an ask.

1

## The task

One thing.  
Name the spec.

2

## The boundary

This branch.  
Never the tests.

3

## The judge

Run the tests  
until they pass.

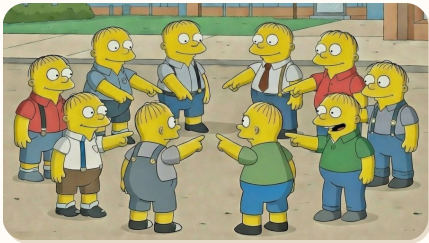
4

## The evidence

The output, and  
every file  
touched.

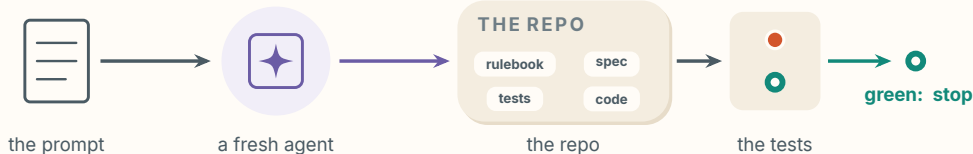
Written out in full on the Week 4 page. Copy it; change two names.

# Ralph.



A cartoon character who is cheerfully, relentlessly wrong, and gets there anyway.  
Not one agent getting better at the task: a crowd of identical beginners, each starting from nothing.

## One run.



A file of instructions. A fresh agent reads it, works in the repo, and the tests say red or green. So far, this is the loop from Act 2, with the four parts of an ask in it.

# Nobody is watching.

```
while ;; do cat PROMPT.md | claude-code ; done
```



Published in 2025, and that is the entire technique. Every run is a new agent, so the repo is the only memory.

# The signs.



## THE PROMPT

One task. Full code, no placeholders. What *done* looks like.

## WHEN IT FAILS

Don't argue with a run. Put a sign in the prompt, and start again.

## THE MEMORY

Spec, plan, rulebook. What it learns goes in a file, or it is gone.

## THE STOP

Tests and a commit, every run. A cap on runs. Then you read it all.

## /goal

```
/goal every test in tests/ passes, and nothing in tests/ changed
```

2026. The shell loop is now one sentence, in the tool.

### WHO DECIDES

After each turn a *second*, smaller model reads the conversation and rules: not yet, done, or impossible. Not the one doing the work.

### WHAT IT CAN SEE

It cannot run your tests. It only reads what the agent put on screen. So the finish line has to be something the agent can show you.

### STILL YOURS

The tool runs the loop now. Nobody has automated deciding what *done* means.

## Feature 2, live.

- 1 **Branch.** Named after the spec.
- 2 **Red first.** See what is missing before anything is written.
- 3 **New thread. Paste the ask.** Then sit on your hands.
- 4 **Watch it loop.**
- 5 **Green — now don't trust it.** Read the diff. Click the app.
- 6 **Commit, push, pull request, merge.**

## One bug, then feature 2.

### SPLITIT

#### Bug 2, then feature 2

Removing a member crashes the page.  
Then expense categories.

### TINY CRM

#### Bug 3, then feature 2

"Won this month" is always zero. Then  
conversion by source.

Files, test names and the ask: Week 4 page.

## When it goes wrong.

- 1 **It edited a test.** Discard. Say the rule. Run again.
- 2 **"Done" — tests red.** Ask for the output.
- 3 **Same failure, twice.** Stop. Read it yourself. New thread.
- 4 **It fixed three other things.** Discard. One task per loop.
- 5 **Green, app wrong.** Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

# Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

## Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

## Rules · judge · loop.